

Lab-5-15_text_classification_pycharm

August 3, 2025

IMT575 Assignment 5.15: Text Classification

Steve Gonzales

1 OVERVIEW

In this lab, you will perform a supervised text regression task. The data for the task will consist of student essays from The William and Flora Hewlett Foundation. The dataset was created to assist in the design of solutions for automated grading of student-written essays. You will use a subset of this dataset and predict the scores of the essays. You may not use external data to make predictions.

```
[151]: from numpy.random import seed
import tensorflow as tf
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
import nltk
from nltk.corpus import stopwords
from sklearn.linear_model import Ridge, Lasso
import tensorflow_hub as hub
```

2 PART 1: SETUP

2.0.1 Q0: Run the following code!

For reproducibility purposes, you will set the random seeds for NumPy and TensorFlow as 1234. This way, all random steps will produce the same answers.

```
[152]: seed(1234)
tf.random.set_seed(seed = 1234)
```

2.0.2 Q1: Load the data

(1 point)

We will use data from the [automated essay scoring task](#) on Kaggle.

We will only use the training data, which we have provided for you – you don't need to download anything from Kaggle.

Access the file `training_set_rel3.tsv` from Files -> Module 5 -> Lab_05_15_Text_Classification -> `training_set_rel3.tsv`

Please load the data from the current directory so that we can run your notebook unmodified.

Use the pandas function `read_csv`, with the parameter `sep='\t'` because this is a tab-separated value file (tsv) and set the parameter `encoding='latin-1'`.

The columns are described on the [Kaggle site](#)

We will use three columns: `essay`, `essay_set`, and `domain1_score`.

Create a new dataframe with only these three columns, and rename `domain1_score` to just `score`.

Display this dataframe.

```
[153]: raw = pd.read_csv("Lab-5-15_training_set_rel3-1.tsv", sep='\t',  
    ↪encoding='latin-1')  
alldata = raw[['essay', 'essay_set', 'domain1_score']].  
    ↪rename(columns={'domain1_score': 'score'})  
display(alldata)
```

	essay	essay_set	score
0	Dear local newspaper, I think effects computer...	1	8
1	Dear @CAPS1 @CAPS2, I believe that using compu...	1	9
2	Dear, @CAPS1 @CAPS2 @CAPS3 More and more peopl...	1	7
3	Dear Local Newspaper, @CAPS1 I have found that...	1	10
4	Dear @LOCATION1, I know having computers has a...	1	8
...	...	...	...
12971	In most stories mothers and daughters are eit...	8	35
12972	I never understood the meaning laughter is th...	8	32
12973	When you laugh, is @CAPS5 out of habit, or is ...	8	40
12974	Trippin' on fen...	8	40
12975	Many people believe that laughter can improve...	8	40

[12976 rows x 3 columns]

2.1 Q2: Select the data from a single essay set

(1 point)

There are 8 totally unrelated essay sets in this data.

Filter the data frame so we are only considering `essay_set = 7`

```
[154]: data = alldata[alldata['essay_set'] == 7]  
print(f"Min score in train: {data['score'].min()} Max score in train: {  
    ↪{data['score'].max()}" )
```

```
display(data)
print(data.shape)
```

Min score in train: 2 Max score in train: 24

	essay	essay_set	score
10684	Patience is when your waiting .I was patience ...	7	15
10685	I am not a patience person, like I cant sit i...	7	13
10686	One day I was at basketball practice and I was...	7	15
10687	I going to write about a time when I went to t...	7	17
10688	It can be very hard for somebody to be patient...	7	13
...	...	...	...
12248	One time I was getting a cool @CAPS1 game it w...	7	12
12249	A patent person in my life is my mom. Aicason ...	7	16
12250	A time when someone else I know was patient wa...	7	19
12251	I hate weddings. I love when people get marrie...	7	22
12252	A few weeks ago, we had a garage sale and a mo...	7	15

[1569 rows x 3 columns]

(1569, 3)

2.1.1 Q3: Plot the distribution of scores

(3 points)

Create a plot of a histogram of the scores in the training set. Comment on what you see.

One option is to use the [seaborn](#) histplot function. If you use seaborn, you can use the parameter `bins` to set the bin locations if they look strange. The parameter accepts a list of explicit locations. If you want to center the bins on the tick marks, you can do something like this:

```
bins=np.arange(minv,maxv)-0.5
```

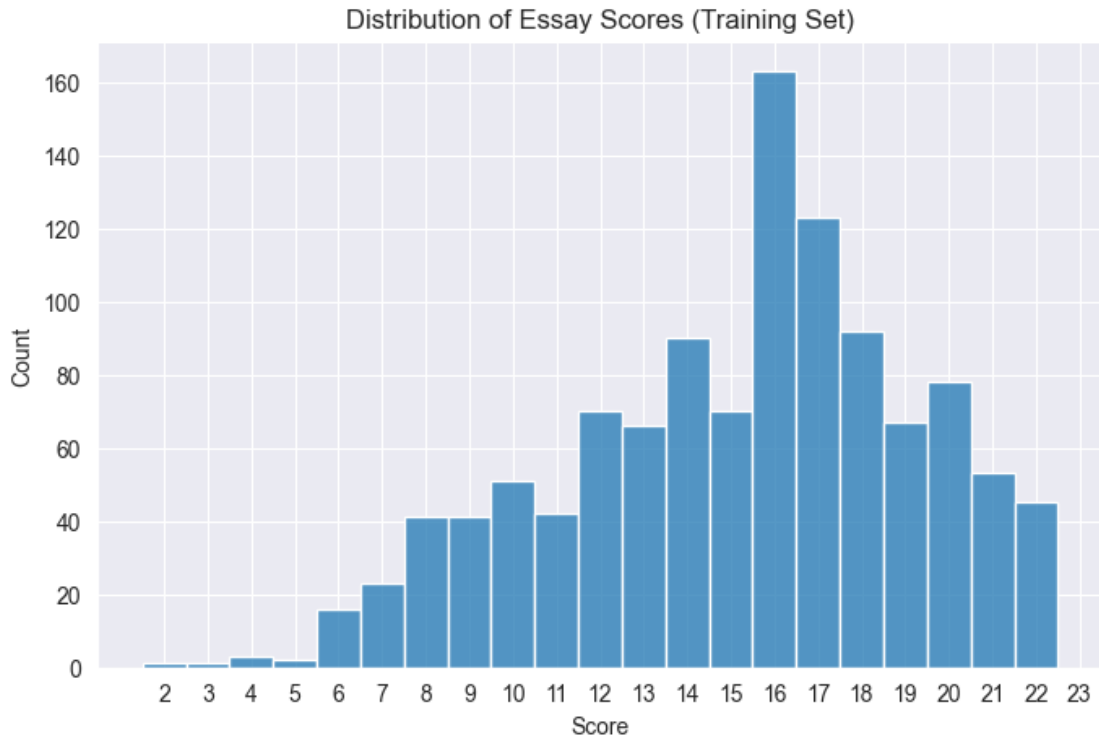
where `minv` and `maxv` are the minimum and maximum value in the range, respectively. This expression indicates the number of possible scores, and that the tick marks should be at the halfway mark of each bar.

You may use some other visualization library if you wish! The goal is to inspect the distribution of scores.

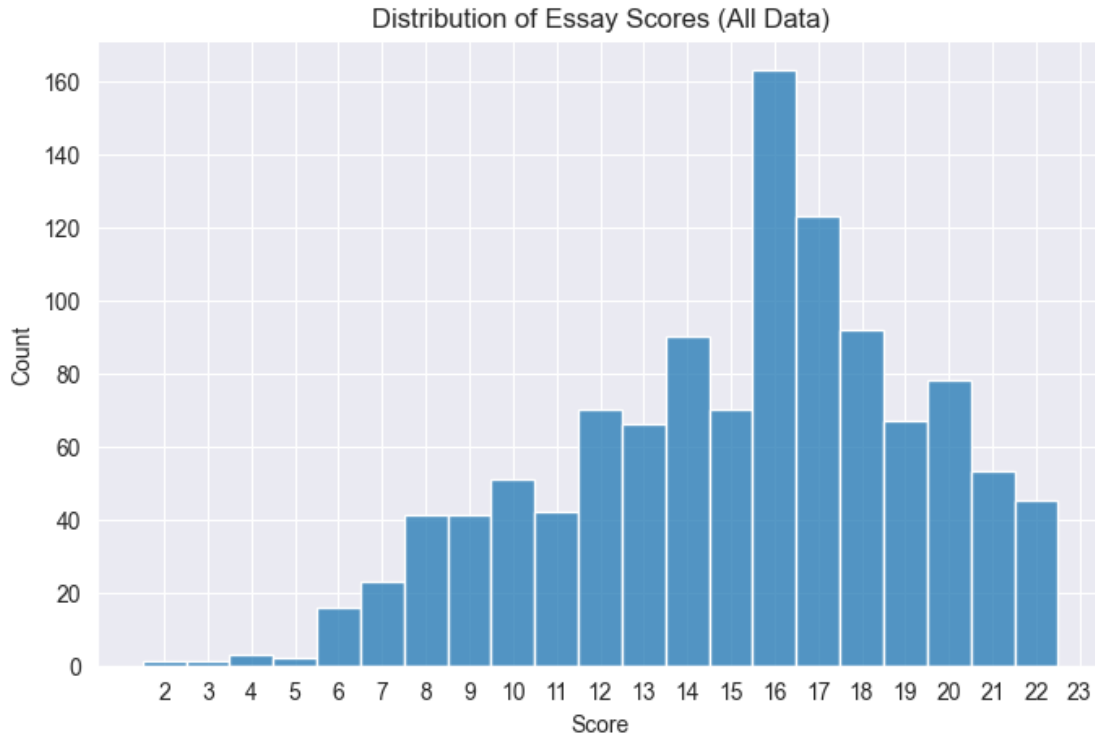
```
[155]: # Instructions say "the training set", I will do both because it doesn't make
      ↪ sense here with the next question.
train, test = train_test_split(data, test_size=0.2, random_state=1234)
minv = train['score'].min()
maxv = train['score'].max()
bins = np.arange(minv, maxv) - 0.5

plt.figure(figsize=(8, 5))
sns.histplot(train['score'], bins=bins, kde=False)
plt.xlabel('Score')
```

```
plt.ylabel('Count')
plt.title('Distribution of Essay Scores (Training Set)')
plt.xticks(np.arange(minv, maxv))
plt.show()
```



```
[156]: # All Data
minv = data['score'].min()
maxv = data['score'].max()
bins = np.arange(minv, maxv) - 0.5
plt.figure(figsize=(8, 5))
sns.histplot(train['score'], bins=bins, kde=False)
plt.xlabel('Score')
plt.ylabel('Count')
plt.title('Distribution of Essay Scores (All Data)')
plt.xticks(np.arange(minv, maxv))
plt.show()
```



I did both full and training data and the histograms look virtually identical.

This histogram shows the distribution of essay scores in the training set. The x-axis represents the test score, while the y-axis represents the count of essays that received each score.

The distribution appears to be somewhat uniform, with a peak around the middle scores but skewed a little towards the higher end. This suggests that most essays received average scores, while fewer essays received very low or very high scores. The range of scores is from 2 to 22, with each score having a similar number of essays assigned to it.

2.1.2 Q4: Create a test/train split

(1 point)

Use the function `train_test_split`. Use the `test_size` parameter to control the size of the test set; use 0.2 to indicate a 20% split.

```
[157]: train, test = train_test_split(data, test_size=0.2, random_state=1234)
print(f"Min score in train: {train['score'].min()} Max score in train: {train['score'].max()}")
display(train)
```

Min score in train: 2 Max score in train: 24

	essay	essay_set	score
11674	There is only one time a year I'm patient for ...	7	13

10817	This is a story about one time I when my mom w...	7	9
12178	AHH! Oh hi my name is @CAPS1 I'm going to tell...	7	24
11236	My mom woke me up early in the morning and sai...	7	18
11496	I was very patient last time I went to the doc...	7	12
...	...	...	...
11912	When I was patient, we were on one way to @CAPS...	7	16
11761	I have to be patient when I'm taking the @CAPS...	7	11
12002	One time when I was patient was when I was in ...	7	15
11407	Being patient is a good thing. Sometimes impat...	7	9
11499	Patience, if you are a patient person you are ...	7	18

[1255 rows x 3 columns]

2.2 Part 2: Conventional Representations

2.2.1 Q5: Create vectors using term frequency

(3 points)

Use the `CountVectorizer` class from `sklearn` to create a vector for each essay. We can't use text directly with machine learning; we need to create a vector of numbers first. The `CountVectorizer` creates a vector with one position for each word in the corpus with a value of the number of occurrences of that word in the essay.

The vectorizer works like a model in `sklearn`: call the `fit` method on the essay data to “train” a model on the training set. In this situation, we aren't really training anything, but we need a corpus to define the vectors – only the words in the corpus we use will be represented in the vector.

The `fit` method returns the trained model. Now we can use the `transform` method to convert any text into a vector.

Call the `transform` method on the training essays and the test essays to create variables `xtrain` and `ytrain`.

Report the number of dimensions for each vector; i.e., the number of terms in the corpus.

```
[158]: cv = CountVectorizer()
cv.fit(train['essay'])
xtrain = cv.transform(train['essay'])
ytrain = train['score']
xtest = cv.transform(test['essay'])
ytest = test['score']
print(xtrain.shape)
print(xtest.shape)
```

(1255, 8938)

(314, 8938)

2.2.2 Q6: Train a regression model using your vectors

(2 point)

Now that we have vectors, we can train a regression model to predict the essay score.

Use a **Ridge** model.

Call the fit method on your training data xtrain and ytrain.

Then call the score method on your test data xtest and ytest. The score method provides a default evaluation metric. For the Ridge model, the score method returns R^2 which is called the coefficient of determination. It tells you the proportion of the variation in the essay score is predictable from the essay text: higher is better.

Report the coefficient of determination.

```
[159]: ridge_basic = Ridge()
       ridge_basic.fit(xtrain, ytrain)
       cs = ridge_basic.score(xtest, ytest)
       print(f"Coefficient of determination R2: {cs:.3f}")
```

Coefficient of determination R2: 0.154

This is a very low score, indicating that the model is not able to predict the essay scores well based on the term frequency vectors.

2.2.3 Q7: Plot the distribution of scores

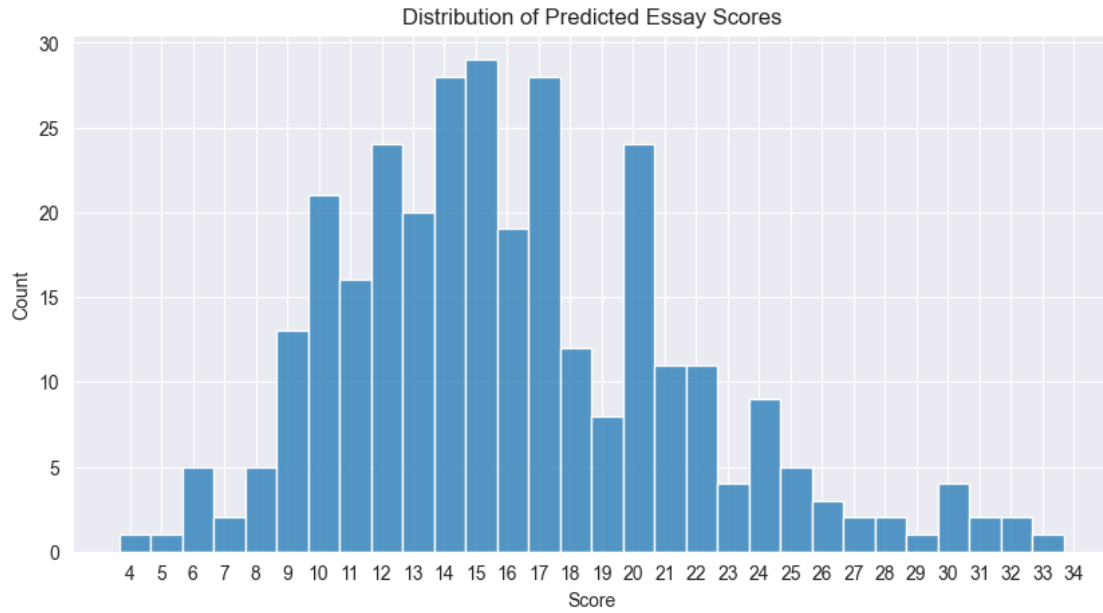
(2 points)

Plot a histogram of your predicted scores.

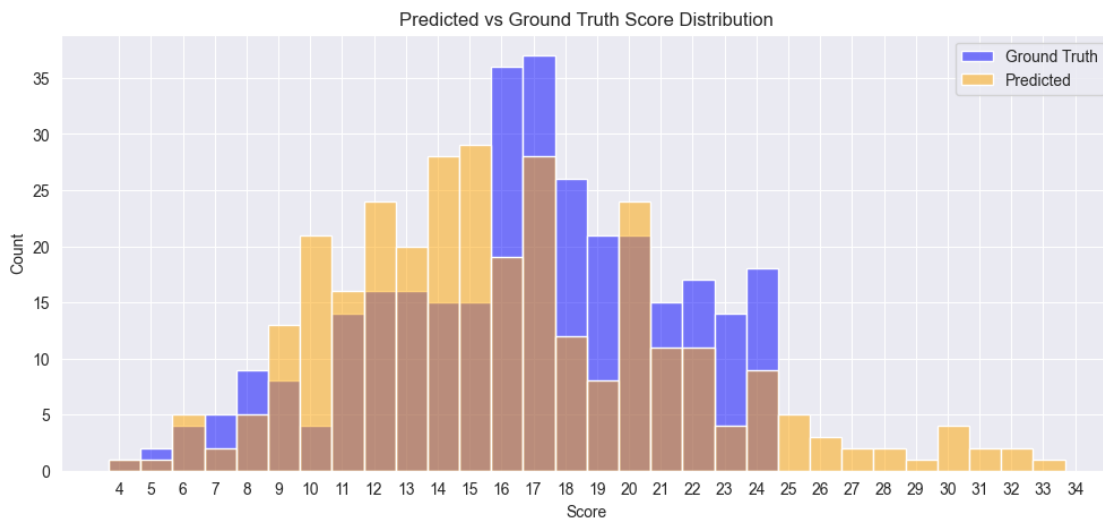
Plot another histogram of the ground truth scores, superimposed on the first (using seaborn, just call the function again.)

How is your model's distribution of scores different from the ground truth distribution? Describe how they differ; what kind of mistakes is your model making?

```
[160]: y_pred = ridge_basic.predict(xtest)
       minv, maxv = y_pred.min(), y_pred.max()
       bins = np.arange(minv, maxv) - 0.5
       plt.figure(figsize=(10, 5))
       sns.histplot(y_pred, bins=bins, kde=False)
       plt.xlabel('Score')
       plt.ylabel('Count')
       plt.title('Distribution of Predicted Essay Scores')
       plt.xticks(np.arange(int(minv), int(maxv) + 1, 1))
       plt.show()
```



```
[161]: bins = np.arange(minv, maxv) - 0.5
plt.figure(figsize=(12, 5))
sns.histplot(ytest, bins=bins, color='blue', label='Ground Truth', alpha=0.5)
sns.histplot(y_pred, bins=bins, color='orange', label='Predicted', alpha=0.5)
plt.xlabel('Score')
plt.ylabel('Count')
plt.title('Predicted vs Ground Truth Score Distribution')
plt.legend()
plt.xticks(np.arange(int(minv), int(maxv) + 1, 1))
plt.show()
```



The predicted scores are way off. They go all the way up to 34, while the ground truth scores only go up to 22. Even for the predictions that fall within the range to 22, the predicted scores are skewed more heavily to lower scores, while the ground truth scores are more concentrated around the higher scores.

2.2.4 Q8: Train a new model using TF-IDF instead of term frequencies

(1 points)

Repeat the last two steps but use the `TfidfVectorizer` class instead of the `CountVectorizer` class. TF-IDF stands for Term Frequency Inverse Document Frequency: this class creates a vector with one position for each word in the corpus with a value of the number of occurrences of that word in the essay (term frequency) divided by (i.e., normalized by) the number of documents in which that term appears (inverse document frequency). (Why do we normalize by document frequency?)

You can pass in a set of stopwords to prevent those words from being used as part of the corpus. The library `nltk` includes a set of stopwords.

Report the score of a Ridge model trained using these vectors. Does the performance improve? Why?

```
[162]: tv = TfidfVectorizer()
tv.fit(train['essay'])
xtrain = tv.transform(train['essay'])
ytrain = train['score']
xtest = tv.transform(test['essay'])
ytest = test['score']
print(xtrain.shape)
print(xtest.shape)

ridge_tfidf = Ridge()
ridge_tfidf.fit(xtrain, ytrain)
cs = ridge_tfidf.score(xtest, ytest)
print(f"Coefficient of determination R2: {cs:.3f}")
y_pred_best_conventional_ridge_tfidf = ridge_tfidf.predict(xtest)
```

```
(1255, 8938)
```

```
(314, 8938)
```

```
Coefficient of determination R2: 0.516
```

Why do we normalize by document frequency?

By Normalizing by document frequency, we reduce the impact of common words that appear in many documents, which may not be as informative for distinguishing between different essays. This helps to highlight more unique and relevant terms that contribute to the meaning of the text, leading to better performance in tasks like essay scoring.

Does the performance improve with Term Frequency Inverse Document Frequency?

Absolutely, the model performs much better with Term Frequency Inverse Document Frequency

(TF-IDF) compared to just using term frequencies. The coefficient of determination (R^2) is significantly higher ~52%, indicating that the model can explain more of the variance in the essay scores. This is likely because TF-IDF takes into account the importance of terms relative to their frequency across documents, which helps to better capture the nuances of the text.

2.2.5 Q9: Plot the distribution of scores

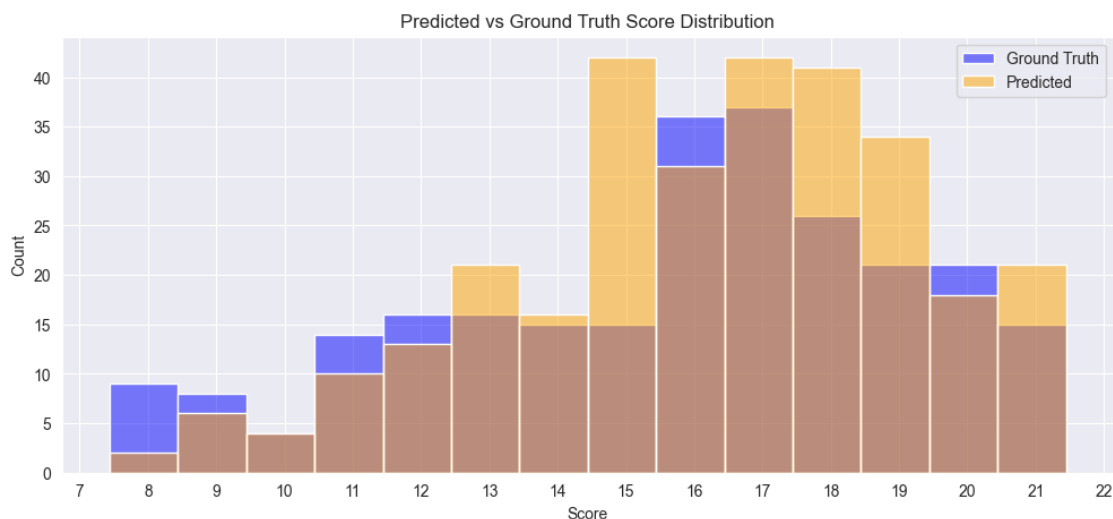
(1 point)

Plot a histogram of your predicted scores from your new model.

Plot another histogram of the ground truth scores.

How is your new model's distribution of scores different from the ground truth distribution? Is it doing better than your first model? How is it doing better?

```
[163]: y_pred = ridge_tfidf.predict(xtest)
minv, maxv = y_pred.min(), y_pred.max()
bins = np.arange(minv, maxv) - .5
plt.figure(figsize=(12, 5))
sns.histplot(ytest, bins=bins, color='blue', label='Ground Truth', alpha=0.5)
sns.histplot(y_pred, bins=bins, color='orange', label='Predicted', alpha=0.5)
plt.xlabel('Score')
plt.ylabel('Count')
plt.title('Predicted vs Ground Truth Score Distribution')
plt.legend()
plt.xticks(np.arange(int(minv), int(maxv) + 1, 1))
plt.show()
```



This model absolutely performs better than the first model. The predicted scores are much closer to the ground truth scores, and the distribution of predicted scores is more aligned with the distribution of ground truth scores. The predicted scores are now mostly within the range of 2 to 22, which is

the range of the ground truth scores. The model is still slightly skewed towards lower scores, but it is a significant improvement over the previous model.

2.2.6 Q10: Try to improve on your model

(5 points)

Try to find a model that produces a better score. Some ideas to consider:

- Use the `ngram_range` parameter of the vectorizer to consider bi-grams (two-term combinations) or tri-grams (three term combinations)
- Use the `stopwords` parameter to customize the set of words to ignore when using the corpus. The library `nltk` includes a set of stopwords in the `corpus` module.
- Use the `alpha` parameter of the Ridge model to change the strength of regularization.
- Use a different linear model, e.g., [Lasso](#)

Report the score (and make sure the score is still R^2 so it can be compared with your previous score).

Once again, include a plot of the histogram of scores to get a sense of what kind of mistakes your model is making.

```
[164]: # Download stopwords
nltk.download('stopwords')
stop_words = stopwords.words('english')

# Use bi-grams and remove stopwords
vectorizer = TfidfVectorizer(ngram_range=(1,2), stop_words=stop_words)
vectorizer.fit(train['essay'])
xtrain = vectorizer.transform(train['essay'])
xtest = vectorizer.transform(test['essay'])
ytrain = train['score']
ytest = test['score']

# Ridge regression with tuned alpha
ridge_tuned = Ridge(alpha=.1)
ridge_tuned.fit(xtrain, ytrain)
ridge_tuned_r2 = ridge_tuned.score(xtest, ytest)
print(f"Ridge R2: {ridge_tuned_r2:.3f}")

# Lasso regression
lasso = Lasso(alpha=0.01, max_iter=10000)
lasso.fit(xtrain, ytrain)
lasso_r2 = lasso.score(xtest, ytest)
print(f"Lasso R2: {lasso_r2:.3f}")
```

```
[nltk_data] Downloading package stopwords to
[nltk_data] /Users/stevegon/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

Ridge R2: 0.445

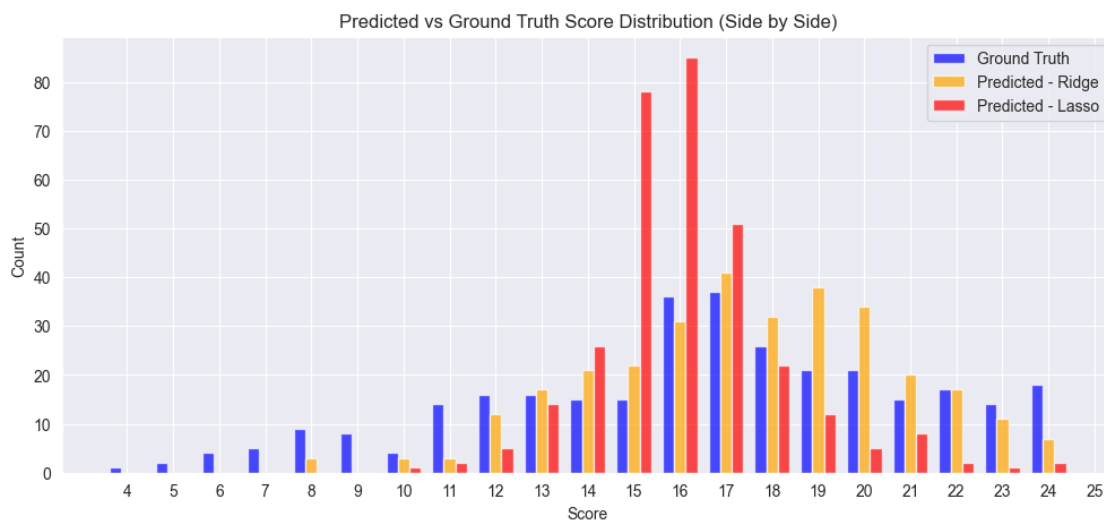
Lasso R2: 0.155

```
[165]: y_pred_ridge = ridge_tuned.predict(xtest)
y_pred_lasso = lasso.predict(xtest)
minv, maxv = int(min(ytest.min(), y_pred_ridge.min(), y_pred_lasso.min())),
↳ int(max(ytest.max(), y_pred_ridge.max(), y_pred_lasso.max())) + 1
bins = np.arange(minv, maxv) - 0.5

# Calculate histogram counts for each group
counts_test, _ = np.histogram(ytest, bins=bins)
counts_ridge, _ = np.histogram(y_pred_ridge, bins=bins)
counts_lasso, _ = np.histogram(y_pred_lasso, bins=bins)

x = bins[:-1] + 0.5 # Center of each bin
width = 0.25

plt.figure(figsize=(12, 5))
plt.bar(x - width, counts_test, width=width, color='blue', label='Ground_
↳ Truth', alpha=0.7)
plt.bar(x, counts_ridge, width=width, color='orange', label='Predicted -
↳ Ridge', alpha=0.7)
plt.bar(x + width, counts_lasso, width=width, color='red', label='Predicted -
↳ Lasso', alpha=0.7)
plt.xlabel('Score')
plt.ylabel('Count')
plt.title('Predicted vs Ground Truth Score Distribution (Side by Side)')
plt.legend()
plt.xticks(np.arange(minv, maxv, 1))
plt.show()
```



Neither Ridge (modified) nor Lasso models improved the R2 score compared to the previous TF-IDF model. The Ridge model achieved an R2 of 0.445, while the Lasso model achieved an R2 of 0.155.

The Modified Ridge model with bi-grams and stopwords came pretty close in the high frequency bins, but could not predict low scores and predicted too many high scores (continuing this pattern from previous models).

2.3 Part 3: Neural Network Representations

For this part, we will implement a deep sentence embedder to replace the feature selection process. As a first step, choose your best-performing model from Part 2.

This time, you will obtain vectors by using a pre-trained neural network model called the Universal Sentence Encoder. This model will produce a dense vector from any sequence of text. (What do I mean by dense?)

First, import the model with the following code. This step will take considerable time – it is downloading a large pre-trained model for the first time.

```
[166]: use_module_url = "https://tfhub.dev/google/universal-sentence-encoder/4"
       use_model = hub.load(use_module_url)
```

This model created a dense vector for each string of text. A dense vector is a vector where most of the values are non-zero, as opposed to a sparse vector where most of the values are zero.

2.3.1 Q11: Generate embeddings

(1 point)

Next, you will embed the data with the imported model. The Universal Sentence Encoder takes a list of strings and generates an embedding (i.e., a vector) for each string.

You can call the model you downloaded like a function.

Generate a vector for each string in the training set; call this array xtrain.

Also generate a vector for each string in the test set; call this array xtest.

Notice how long this step takes – it's a big model.

```
[167]: # Generate embeddings for training and test sets
xtrain = use_model(list(train['essay'])).numpy()
xtest = use_model(list(test['essay'])).numpy()
print(xtrain[:5])
print(xtest[:5])
```

```
[[-0.00058366 -0.06227732 -0.05050291 ... -0.02245546 -0.05134018
  -0.05642162]
 [ 0.02360792 -0.03169743  0.06948538 ...  0.04035845 -0.04671663
  -0.00656356]
 [-0.04850439 -0.02876119 -0.01251371 ...  0.03231099  0.02412072
```

```

-0.05395402]
[-0.02404284 -0.01079837  0.06139803 ... -0.00663147 -0.01519066
-0.00491238]
[-0.08467614 -0.03363819 -0.01522398 ... -0.00802465  0.0018337
-0.00583781]]
[[-0.04434904 -0.01502894  0.00990444 ... -0.00617089  0.04016917
-0.04140122]
[-0.06715522 -0.01980796 -0.00361663 ... -0.01996366 -0.03966582
-0.03479327]
[ 0.01372852 -0.05549547 -0.070877    ... -0.04431206 -0.05382478
-0.01894062]
[-0.02329617 -0.00297889 -0.01723766 ... -0.01522436  0.02728785
-0.0430154 ]
[-0.03206076 -0.01072412 -0.00591833 ... -0.04593699  0.01832767
 0.00609922]]

```

2.3.2 Q12: Train and evaluate a regression model to predict scores using learned embeddings

(1 point)

Now retrain your regression model on these learned embeddings instead of the tfidf vectors.

Use the vanilla Ridge model. Report the score.

Which model appears to perform the best?

```

[168]: ytrain = train['score'].values
       ytest = test['score'].values

       ridge_use = Ridge(alpha=1.0)
       ridge_use.fit(xtrain, ytrain)
       cs = ridge_use.score(xtest, ytest)
       print(f"Coefficient of determination R2: {cs:.3f}")

```

Coefficient of determination R2: 0.629

This model is the best model so far, with an R2 score of 0.629. This indicates that the model can explain in excess of 67% of the variance in the essay scores based on the learned embeddings from the Universal Sentence Encoder. The embeddings capture semantic meaning and context better than the previous term frequency or TF-IDF models, leading to improved performance.

2.3.3 Q13: Plot the distribution of scores

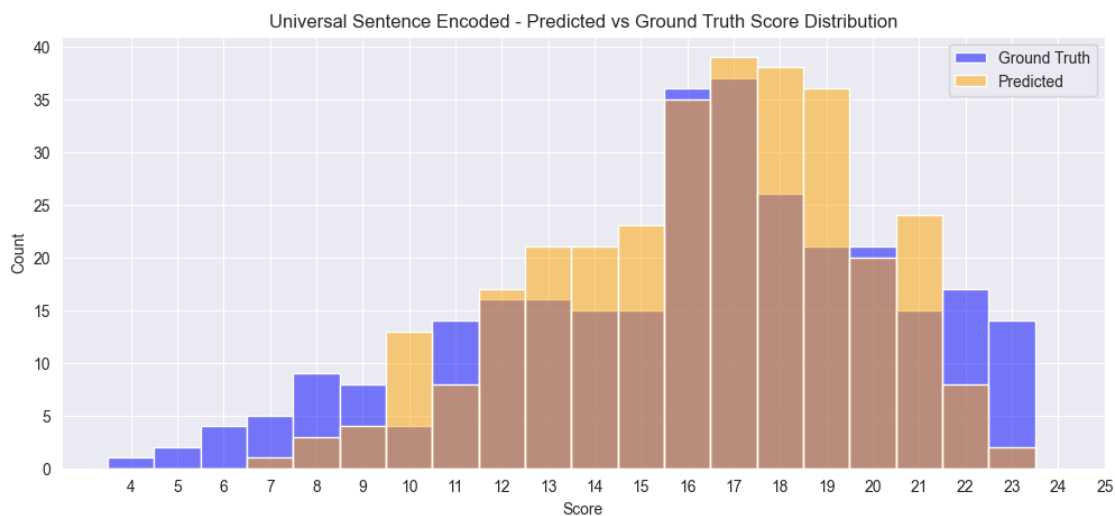
(1 point)

Once again, plot a histogram of your predicted scores from your new model.

Plot another histogram of the ground truth scores.

How is your new model's distribution of scores different from the ground truth distribution? Is it doing better than your earlier models? How is it doing better?

```
[169]: y_pred = ridge_use.predict(xtest)
minv, maxv = min(ytest.min(), y_pred.min()), max(ytest.max(), y_pred.max()) + 1
bins = np.arange(minv, maxv) - 0.5
plt.figure(figsize=(12, 5))
sns.histplot(ytest, bins=bins, color='blue', label='Ground Truth', alpha=0.5)
sns.histplot(y_pred, bins=bins, color='orange', label='Predicted', alpha=0.5)
plt.xlabel('Score')
plt.ylabel('Count')
plt.title('Universal Sentence Encoded - Predicted vs Ground Truth Score_
↪Distribution')
plt.legend()
plt.xticks(np.arange(int(minv), int(maxv) + 1, 1))
plt.show()
```



2.3.4 Q14: Plot the errors

(1 point)

We will analyze the difference between the neural model and your best conventional model.

Plot the distribution of errors – see where the two models made mistakes.

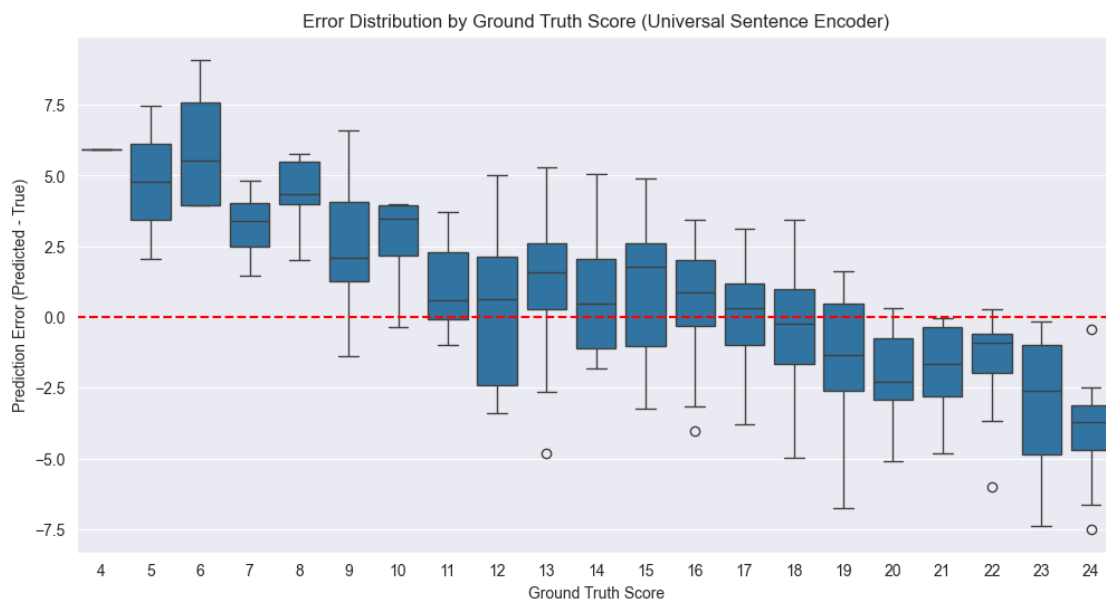
The errors are your model's predicted score minus the ground truth human score.

Plot a boxplot of the errors for your model using the universal sentence encoder. Use the seaborn histplot function.

x will be the ground truth scores and y is the difference between ground truth and your predictions.

What do you see? Where does the model tend to make mistakes?

```
[170]: # Calculate errors (Prediction - Truth)
errors_use = y_pred - ytest
# Create a DataFrame for plotting
df_errors_use = pd.DataFrame({'Score': ytest, 'Error': errors_use})
plt.figure(figsize=(12, 6))
sns.boxplot(x='Score', y='Error', data=df_errors_use)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Ground Truth Score')
plt.ylabel('Prediction Error (Predicted - True)')
plt.title('Error Distribution by Ground Truth Score (Universal Sentence Encoder)')
plt.show()
```



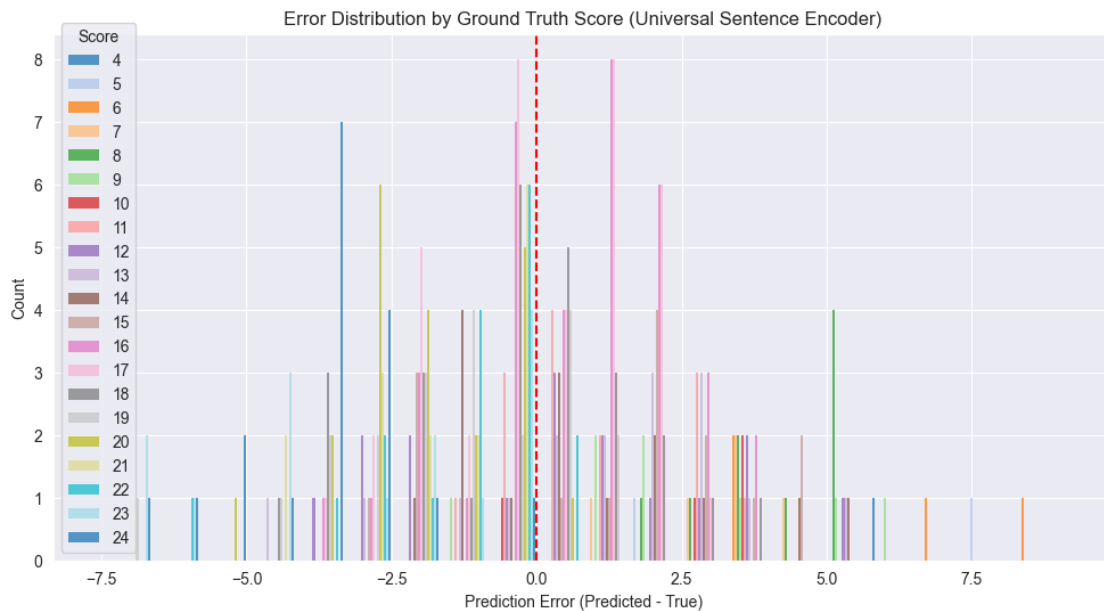
The BoxPlot tells us that many of the errors are between 2 ~ 5 points, with a few outliers that are much larger. The model tends to overestimate the scores, as indicated by the positive errors. The median error is slightly positive, suggesting that the model generally predicts higher scores than the ground truth. There are also some negative errors, indicating underestimation, but they are less frequent and less extreme than the overestimations.

Reference to the above BoxPlot: - Top horizontal line (upper whisker): The largest data point within $1.5 \times \text{IQR}$ above the 75th percentile (Q3). - Bottom horizontal line (lower whisker): The smallest data point within $1.5 \times \text{IQR}$ below the 25th percentile (Q1). - Rectangle (box): Spans from the 25th percentile (Q1, bottom of box) to the 75th percentile (Q3, top of box); this is the interquartile range (IQR). - Line inside rectangle: The median (50th percentile) of the data. - Small circles: Outliers, which are data points that fall outside the whiskers.

```
[171]: #Histplot; although this is different than in the homework template which shows a
        ↪ a boxplot/candlestick plot
```



```
plt.figure(figsize=(12, 6))
sns.histplot(data=df_errors_use, x='Error', hue='Score', multiple='dodge',
             bins=20, palette='tab20', edgecolor='none')
plt.axvline(0, color='red', linestyle='--')
plt.xlabel('Prediction Error (Predicted - True)')
plt.ylabel('Count')
plt.title('Error Distribution by Ground Truth Score (Universal Sentence
         Encoder)')
plt.show()
```



I've included this chart as per the instructions, but it is not as useful as the boxplot. The boxplot provides a clearer view of the distribution of errors across different ground truth scores, while the histogram shows the overall distribution of errors without the context of the ground truth scores. The boxplot allows us to see how the model's predictions vary with respect to the actual scores, which is more informative for understanding model performance.

2.3.5 Q15: Compare models directly

(1 point)

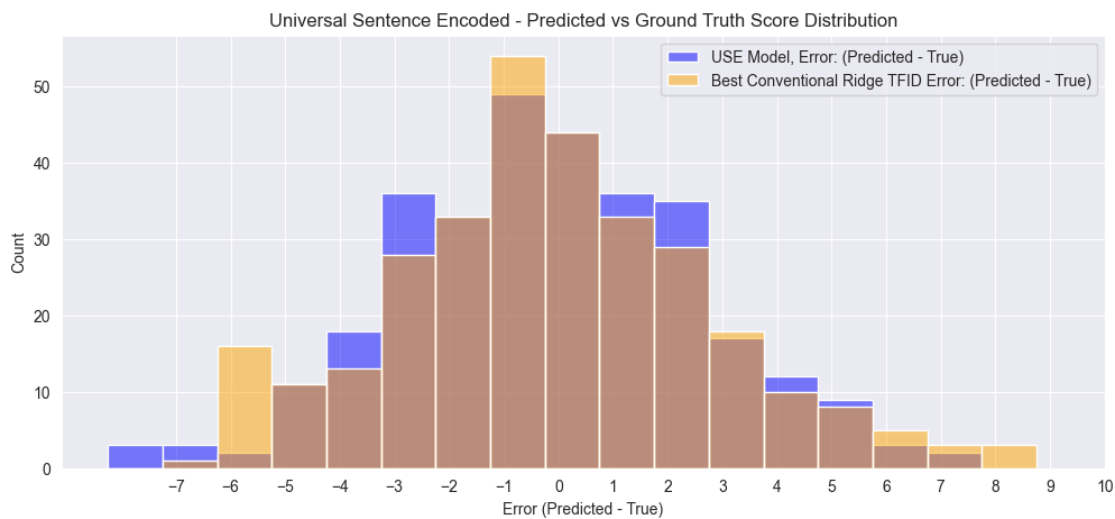
Plot a histogram of the difference between your neural model and the ground truth.

Plot another histogram of the difference between your best conventional model and the ground truth.

Does either model tend to overestimate or underestimate the true score?

```
[172]: errors_best_conventional_ridge_tfidf = y_pred_best_conventional_ridge_tfidf -
    ↳ ytest
df_errors_best_conventional = pd.DataFrame({'Score': ytest, 'Error':
    ↳ errors_best_conventional_ridge_tfidf})

minv, maxv = df_errors_use["Error"].min(), df_errors_use["Error"].max() + 1
bins = np.arange(minv, maxv) - .75
plt.figure(figsize=(12, 5))
sns.histplot(df_errors_use["Error"], bins=bins, color='blue', label='USE Model,
    ↳ Error: (Predicted - True)', alpha=0.5)
sns.histplot(df_errors_best_conventional["Error"], bins=bins, color='orange',
    ↳ label='Best Conventional Ridge TFID Error: (Predicted - True)', alpha=0.5)
plt.xlabel('Error (Predicted - True)')
plt.ylabel('Count')
plt.title('Universal Sentence Encoded - Predicted vs Ground Truth Score
    ↳ Distribution')
plt.legend()
plt.xticks(np.arange(int(minv), int(maxv) + 1, 1))
plt.show()
```



The Universal Sentence Encoder model tends to underestimate the true scores more, as indicated by a few more negative errors than the Ridge TF-IDF model.

The Ridge TF-IDF model tends to overestimate the true scores more, as indicated by a few more positive errors than the Universal Sentence Encoder model.

2.3.6 Q14: Summarize your findings

(5 points)

Summarize your results. Which approach worked best? Why? Does automatic essay scoring

appear feasible? What are some risks or ethical issues with automating this process? How might we improve on this model?

Basic Ridge with The Universal Sentence Encoder achieved the highest R2 score of 0.629, indicating that it can explain about 63% of the variance in essay scores based on the learned embeddings. This model outperformed all conventional models, including those using TF-IDF and term frequency vectors.

However, this is absolutely not good enough for automatic essay scoring. The model still makes significant errors, and the distribution of predicted scores does not align well with the ground truth distribution.

In my experience working on highly profitable predictive models, we generally consider a model successful above 85% accuracy and if a model falls below 85% we start to investigate why it is happening. If the model consistently performs below 80% accuracy, it is taken offline or replaced with a model that performs better. In this case, the model is not even close to being good enough for production use.

How might we improve on this model?

The simplest way to improve on this model, assuming we have the data would be to include essay length and grammar/spelling error counts.

Will try an RNN model next, which should be able to learn more complex relationships in the data:

```
[173]: from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Prepare data
max_words = 10000
max_len = 800

tokenizer = Tokenizer(num_words=max_words, oov_token='<OOV>')
tokenizer.fit_on_texts(train['essay'])
xtrain_seq = tokenizer.texts_to_sequences(train['essay'])
xtest_seq = tokenizer.texts_to_sequences(test['essay'])
xtrain_pad = pad_sequences(xtrain_seq, maxlen=max_len, padding='post')
xtest_pad = pad_sequences(xtest_seq, maxlen=max_len, padding='post')
ytrain = train['score'].values
ytest = test['score'].values

# Build RNN model
model = Sequential([
    Embedding(input_dim=max_words, output_dim=128),
    Bidirectional(LSTM(64, return_sequences=False)),
    Dropout(0.3),
```

```

        Dense(32, activation='relu'),
        Dense(1) # Regression output
    ])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()

# Train
early_stop = EarlyStopping(monitor='val_loss', patience=3,
    ↳ restore_best_weights=True)
model.fit(xtrain_pad, ytrain, epochs=5, batch_size=32, validation_split=0.2,
    ↳ callbacks=[early_stop])

# Evaluate
loss, mae = model.evaluate(xtest_pad, ytest)
print(f"Test MAE: {mae:.3f}")

# Predict and calculate R2
from sklearn.metrics import r2_score
y_pred = model.predict(xtest_pad).flatten()
r2 = r2_score(ytest, y_pred)
print("-"*50)
print(f"R2 Score: {r2:.3f}")

```

Model: "sequential_7"

Layer (type)	Output Shape	Param #
embedding_7 (Embedding)	?	0 (unbuilt)
bidirectional_6 (Bidirectional)	?	0 (unbuilt)
dropout_7 (Dropout)	?	0
dense_14 (Dense)	?	0 (unbuilt)
dense_15 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```

Epoch 1/5
32/32          7s 179ms/step -
loss: 230.3769 - mae: 14.1543 - val_loss: 34.6995 - val_mae: 4.9141
Epoch 2/5
32/32          6s 186ms/step -
loss: 27.9029 - mae: 4.3061 - val_loss: 21.1605 - val_mae: 3.6934
Epoch 3/5
32/32          7s 205ms/step -
loss: 24.5895 - mae: 4.0490 - val_loss: 21.1620 - val_mae: 3.6993
Epoch 4/5
32/32          9s 289ms/step -
loss: 24.4179 - mae: 4.0230 - val_loss: 21.1328 - val_mae: 3.6963
Epoch 5/5
32/32          9s 292ms/step -
loss: 24.0016 - mae: 4.0131 - val_loss: 21.0834 - val_mae: 3.6884
10/10          0s 44ms/step -
loss: 22.6498 - mae: 3.8673
Test MAE: 3.688
10/10          1s 53ms/step
-----
R2 Score: -0.012

```

Even attempting epochs up to 20, the RNN model absolutely performed worse than the others with mostly negative R2 scores. This is likely due to the small dataset size and the complexity of the RNN model, which may not generalize well with limited data. The model may also be overfitting to the training data, leading to poor performance on the test set.

```

[175]: from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D,
↳Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

model = Sequential([
    Embedding(input_dim=max_words, output_dim=128),
    Conv1D(128, 5, activation='relu'),
    GlobalMaxPooling1D(),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dense(1) # Regression output
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()

early_stop = EarlyStopping(monitor='val_loss', patience=3,
↳restore_best_weights=True)

```

```

model.fit(xtrain_pad, ytrain, epochs=10, batch_size=32, validation_split=0.2,
        ↪callbacks=[early_stop])

# Evaluate
loss, mae = model.evaluate(xtest_pad, ytest)
print(f"Test MAE: {mae:.3f}")

from sklearn.metrics import r2_score
y_pred = model.predict(xtest_pad).flatten()
r2 = r2_score(ytest, y_pred)
print("-"*50)
print(f"R2 Score: {r2:.3f}")

```

Model: "sequential_8"

Layer (type)	Output Shape	Param #
embedding_8 (Embedding)	?	0 (unbuilt)
conv1d_1 (Conv1D)	?	0 (unbuilt)
global_max_pooling1d_1 (GlobalMaxPooling1D)	?	0
dropout_8 (Dropout)	?	0
dense_16 (Dense)	?	0 (unbuilt)
dense_17 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

32/32 1s 27ms/step -

loss: 249.1424 - mae: 14.8938 - val_loss: 27.1381 - val_mae: 4.3428

Epoch 2/10

32/32 1s 26ms/step -

loss: 32.2109 - mae: 4.5316 - val_loss: 18.4209 - val_mae: 3.4749

Epoch 3/10

```

32/32          1s 27ms/step -
loss: 20.4802 - mae: 3.6539 - val_loss: 18.3847 - val_mae: 3.4322
Epoch 4/10
32/32          1s 27ms/step -
loss: 19.0011 - mae: 3.5711 - val_loss: 17.0592 - val_mae: 3.3099
Epoch 5/10
32/32          1s 27ms/step -
loss: 16.8710 - mae: 3.3463 - val_loss: 15.2008 - val_mae: 3.1301
Epoch 6/10
32/32          1s 27ms/step -
loss: 14.2153 - mae: 3.0487 - val_loss: 14.3853 - val_mae: 3.0481
Epoch 7/10
32/32          1s 26ms/step -
loss: 12.8621 - mae: 2.8921 - val_loss: 13.4219 - val_mae: 2.9531
Epoch 8/10
32/32          1s 27ms/step -
loss: 10.7098 - mae: 2.5981 - val_loss: 12.5773 - val_mae: 2.8619
Epoch 9/10
32/32          1s 27ms/step -
loss: 7.7037 - mae: 2.1919 - val_loss: 11.8508 - val_mae: 2.7933
Epoch 10/10
32/32          1s 27ms/step -
loss: 5.8468 - mae: 1.9084 - val_loss: 10.3584 - val_mae: 2.6154
10/10          0s 8ms/step - loss:
10.6209 - mae: 2.6136
Test MAE: 2.517
10/10          0s 9ms/step
-----
R2 Score: 0.522

```

Without significantly more effort like determining grammatical and spelling errors or using a Fine-Tuned transformer like BERT (which we have not covered in this course), it is unlikely that we will be able to achieve a model that performs well enough for production use. The RNN and CNN models did not perform better than the Ridge model with the Universal Sentence Encoder embeddings, and they are more complex and harder to interpret.